

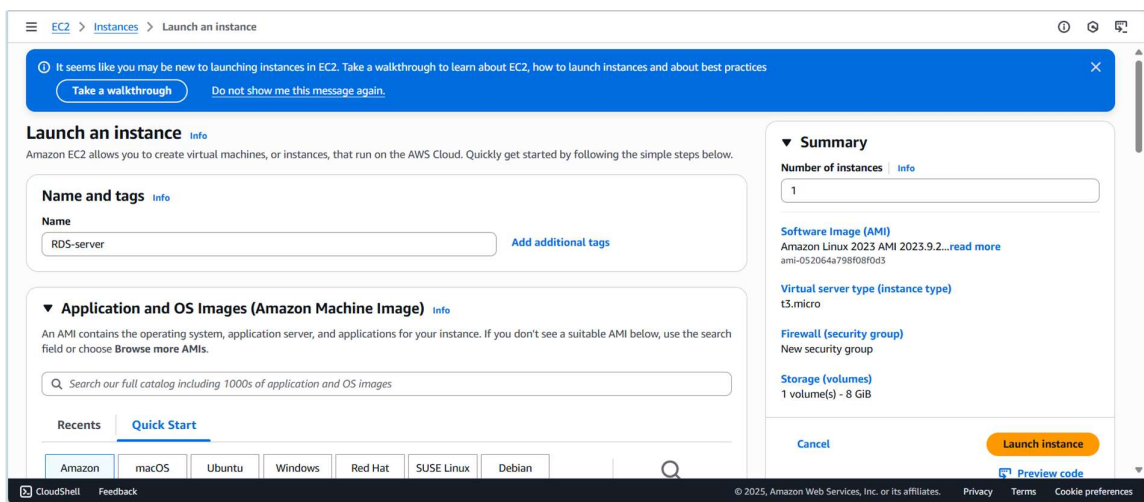
Task on RDS

1. Create a MariaDB Database on EC2.

Purpose: This task involves **setting up a MariaDB database on an Amazon EC2 instance** to host relational data for applications. The **main purpose** is to provide a **customizable, scalable, and cost-effective** database solution, giving you full administrative control over configurations, security, and performance. It's ideal for applications requiring dedicated database management, and it helps you gain hands-on experience with cloud-based database deployment.

Steps to create a MariaDB on EC2:

- Launch an instance and run the following commands to install and start MariaDB.



- `# yum install -y mariadb105-server`

```
[root@ip-172-31-17-251 ec2-user]# yum install -y mariadb105-server
Amazon Linux 2023 Kernel Livepatch repository                238 kB/s | 26 kB    00:00
Dependencies resolved.
=====
Package                                Architecture Version                                Repository          Size
-----
Installing:
mariadb105-server                      x86_64    3:10.5.29-1.amzn2023.0.1             amazonlinux          10 M
Installing dependencies:
mariadb-connector-c                    x86_64    3.3.10-1.amzn2023.0.1                 amazonlinux          211 k
mariadb-connector-c-config              noarch    3.3.10-1.amzn2023.0.1                 amazonlinux          9.9 k
mariadb105                             x86_64    3:10.5.29-1.amzn2023.0.1             amazonlinux          1.5 M
mariadb105-common                      x86_64    3:10.5.29-1.amzn2023.0.1             amazonlinux          28 k
mariadb105-errmsg                      x86_64    3:10.5.29-1.amzn2023.0.1             amazonlinux          212 k
mysql-selinux                          noarch    1.0.4-2.amzn2023.0.3                 amazonlinux          36 k
perl-B                                  x86_64    1.80-477.amzn2023.0.7                 amazonlinux          177 k
perl-DBD-MariaDB                       x86_64    1.22-1.amzn2023.0.4                   amazonlinux          153 k
perl-DBI                               x86_64    1.643-7.amzn2023.0.3                  amazonlinux          700 k
perl-Data-Dumper                       x86_64    2.174-460.amzn2023.0.2                 amazonlinux          55 k
perl-File-Copy                          noarch    2.34-477.amzn2023.0.7                 amazonlinux          20 k
perl-FileHandle                        noarch    2.03-477.amzn2023.0.7                 amazonlinux          15 k
perl-Math-BigInt                       noarch    1:1.9998.39-2.amzn2023.0.2            amazonlinux          202 k
perl-Math-BigRat                       noarch    0.2624-500.amzn2023.0.2               amazonlinux          42 k
perl-Math-Complex                      noarch    1.59-477.amzn2023.0.7                 amazonlinux          46 k
perl-Sys-Hostname                      x86_64    1.23-477.amzn2023.0.7                 amazonlinux          16 k
perl-base                              noarch    2.27-477.amzn2023.0.7                 amazonlinux          16 k
Installing weak dependencies:
```

- *# systemctl enable mariadb*
- *# systemctl start mariadb*
- *# systemctl status mariadb*
- *# yum -y update*

```
Complete!
[root@ip-172-31-17-251 ec2-user]# systemctl enable mariadb
Created symlink /etc/systemd/system/mysql.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/mysqld.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/multi-user.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
[root@ip-172-31-17-251 ec2-user]# systemctl start mariadb
[root@ip-172-31-17-251 ec2-user]# systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-10-13 09:28:58 UTC; 24s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 26515 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
   Process: 26537 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
   Process: 26635 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
  Main PID: 26620 (mariabdb)
    Status: "Taking your SQL requests now..."
   Tasks: 14 (limit: 1053)
  Memory: 66.4M
    CPU: 474ms
   CGroup: /system.slice/mariadb.service
           └─26620 /usr/libexec/mariabdb --basedir=/usr
```

```
[root@ip-172-31-17-251 ec2-user]# lsof -i tcp:3306
COMMAND PID USER   FD  TYPE DEVICE SIZE/OFF NODE NAME
mariabdb 26620 mysql 19u  IPv6 44817      0t0  TCP *:mysql (LISTEN)
[root@ip-172-31-17-251 ec2-user]#
```

```
[root@ip-172-31-28-165 tmp]# sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 9
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| ec2db    |
| information_schema |
| mysql    |
| performance_schema |
+-----+
4 rows in set (0.000 sec)

MariaDB [(none)]>
```

i-Of2ec47bcfb95741b (rds-server)

PublicIPs: 54.90.233.46 PrivateIPs: 172.31.28.165

2. Insert some dummy data.

Purpose: This task involves populating your MariaDB database with sample or dummy data to simulate real-world usage. The main purpose is to test database functionality, validate queries, and ensure your application interacts correctly with the database before deploying it in a production environment. It helps identify issues early, such as schema errors or performance bottlenecks, and provides a realistic dataset for development and debugging.

Steps to Insert Dummy Data:

Step 1: Run the following commands to set the environmental variable.

- *DBName=productiondb*
- *DBPassword=admin123456*
- *DBRootPassword=admin123456*
- *DBUser=syeduser*

```
[root@ip-172-31-17-251 ec2-user]# DBName=productiondb
[root@ip-172-31-17-251 ec2-user]# DBPassword=admin123
[root@ip-172-31-17-251 ec2-user]# DBRootPassword=admin123
[root@ip-172-31-17-251 ec2-user]# DBUser=syeduser
[root@ip-172-31-17-251 ec2-user]#
[root@ip-172-31-17-251 ec2-user]# echo $DBName
productiondb
[root@ip-172-31-17-251 ec2-user]# echo $DBPassword
admin123
[root@ip-172-31-17-251 ec2-user]# echo $DBRootPassword
admin123
[root@ip-172-31-17-251 ec2-user]# echo $DBUser
syeduser
[root@ip-172-31-17-251 ec2-user]# |
```

Step 2: Run the following commands to create a database.

- *# echo "CREATE DATABASE \${DBName};" >> /tmp/db.setup*
- *# echo "CREATE USER '\${DBUser}' IDENTIFIED BY '\${DBPassword}';" >> /tmp/db.setup*
- *# echo "GRANT ALL PRIVILEGES ON *.* TO '\${DBUser}'@'%';" >> /tmp/db.setup*
- *# echo "FLUSH PRIVILEGES;" >> /tmp/db.setup*
- *# mysqladmin -u root password "\${DBRootPassword}"*
- *# mysql -u root --password="\${DBRootPassword}" < /tmp/db.setup*
- *# rm /tmp/db.setup*

```

[root@ip-172-31-17-251 ec2-user]# echo "CREATE DATABASE ${DBName};" >> /tmp/db.setup
[root@ip-172-31-17-251 ec2-user]# echo "CREATE USER '${DBUser}' IDENTIFIED BY '${DBPassword}':" >> /tmp/db.setup
[root@ip-172-31-17-251 ec2-user]# echo "GRANT ALL PRIVILEGES ON *.* TO '${DBUser}'@'%';" >> /tmp/db.setup
[root@ip-172-31-17-251 ec2-user]# echo "FLUSH PRIVILEGES;" >> /tmp/db.setup
bash: /tmp/db.setup: No such file or directory
[root@ip-172-31-17-251 ec2-user]# echo "FLUSH PRIVILEGES;" >> /tmp/db.setup
bash: /tmp/db.setup: No such file or directory
[root@ip-172-31-17-251 ec2-user]# echo "GRANT ALL PRIVILEGES ON *.* TO '${DBUser}'@'%';" >> /tmp/db.setup
[root@ip-172-31-17-251 ec2-user]# echo "FLUSH PEIVILEGES;" >> /tmp/db.setup
[root@ip-172-31-17-251 ec2-user]# echo "FLUSH PRIVILEGES;" >> /tmp/db.setup
[root@ip-172-31-17-251 ec2-user]# mysqladmin -u root password "${DBRootPassword}"
[root@ip-172-31-17-251 ec2-user]# mysql -u root --password="${DBRootPassword}" < /tmp/db.setup
bash: /tmp/db.setup: No such file or directory
[root@ip-172-31-17-251 ec2-user]# mysqladmin -u root password "${DBRootPassword}"
[root@ip-172-31-17-251 ec2-user]# mysql -u root --password="${DBRootPassword}" < /tmp/db.setup
-----
CREATE DATABASE productiondb;
CREATE USER 'syeduser' IDENTIFIED BY 'admin123';
GRANT ALL PRIVILEGES ON *.* TO 'syeduser'@'%';

```

- *mysql -u root --password="\${DBRootPassword}"*
- *show databases;*
- *USE gowthamdb;*
- *CREATE TABLE table1 (id INT, name VARCHAR(45));*
- *INSERT INTO table1 VALUES(1, 'Virat'), (2, 'Sachin'), (3, 'Dhoni'), (4, 'ABD');*
- *SELECT * FROM table1;*

3. Take a backup of dummy data on EC2.

Purpose: This task involves creating a backup of your MariaDB database containing the dummy data on your EC2 instance. The main purpose is to safeguard your data against accidental loss, corruption, or system failures. By taking regular backups, you ensure quick recovery and continuity in case of unexpected issues, while also maintaining a reliable dataset for testing and development. This practice is essential for data integrity and disaster recovery in any production or learning environment.

Steps to take a backup of Dummy Data on EC2:

Step 1: Run the following command to take a backup and store it in a **.sql** file

- `mysql dump -u root -p user_name > file_name.sql`

```
[root@ip-172-31-17-251 ec2-user]# mysql dump -u root -p productiondb > mariadb_data_backup.sql
[root@ip-172-31-17-251 ec2-user]# cat mariadb_data_backup.sql
mysql Ver 15.1 Distrib 10.5.29-MariaDB, for Linux (x86_64) using EditLine wrapper
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Usage: mysql [OPTIONS] [database]

Default options are read from the following files in the given order:
/etc/my.cnf ~/.my.cnf
The following groups are read: mysql mariadb-client client-server client-mariadb
The following options may be given as the first argument:
--print-defaults          Print the program argument list and exit.
--no-defaults             Don't read default options from any option file.
The following specify which files/extra groups are read (specified before remaining options):
--defaults-file=#         Only read default options from the given file #.
--defaults-extra-file=#   Read this file after the global files are read.
--defaults-group-suffix=# Additionally read default groups with # appended as a suffix.
```

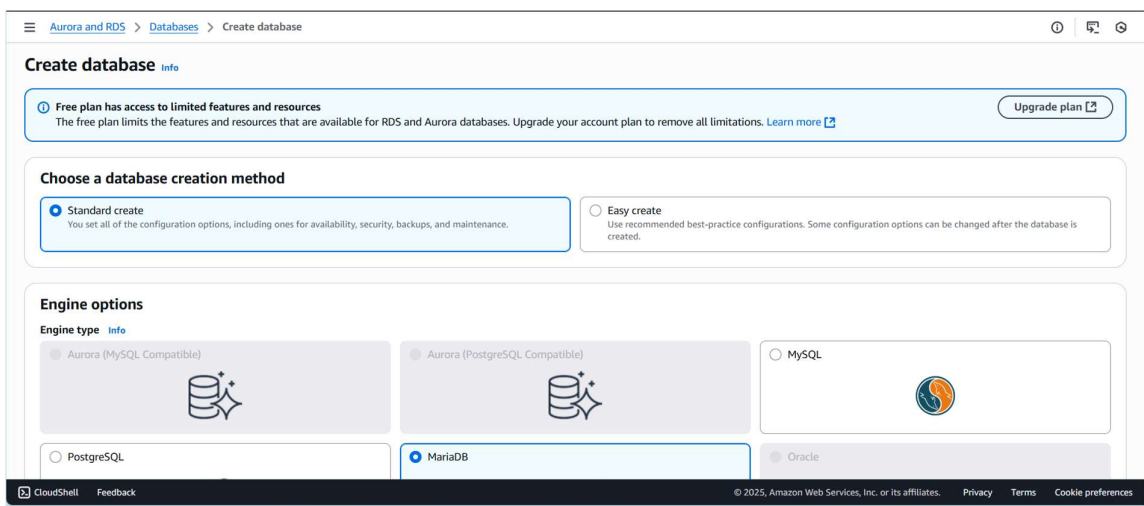
```
ignore            FALSE
socket            (No default value)
ssl               FALSE
ssl-ca            (No default value)
ssl-capath        (No default value)
ssl-cert          (No default value)
ssl-cipher        (No default value)
ssl-key           (No default value)
ssl-crl           (No default value)
ssl-crlpath       (No default value)
tls-version       (No default value)
ssl-verify-server-cert FALSE
table            FALSE
unbuffered        FALSE
user             root
vertical          FALSE
xml              FALSE
[root@ip-172-31-17-251 ec2-user]# ls
mariadb_data_backup.sql
[root@ip-172-31-17-251 ec2-user]#
```

4. Launch MariaDB RDS instance.

Purpose: This task involves creating a managed MariaDB database instance using Amazon RDS (Relational Database Service). The main purpose is to offload database administration tasks—such as backups, patching, and scaling—to AWS, allowing you to focus on application development. RDS provides high availability, automated backups, and seamless scalability, making it ideal for production environments.

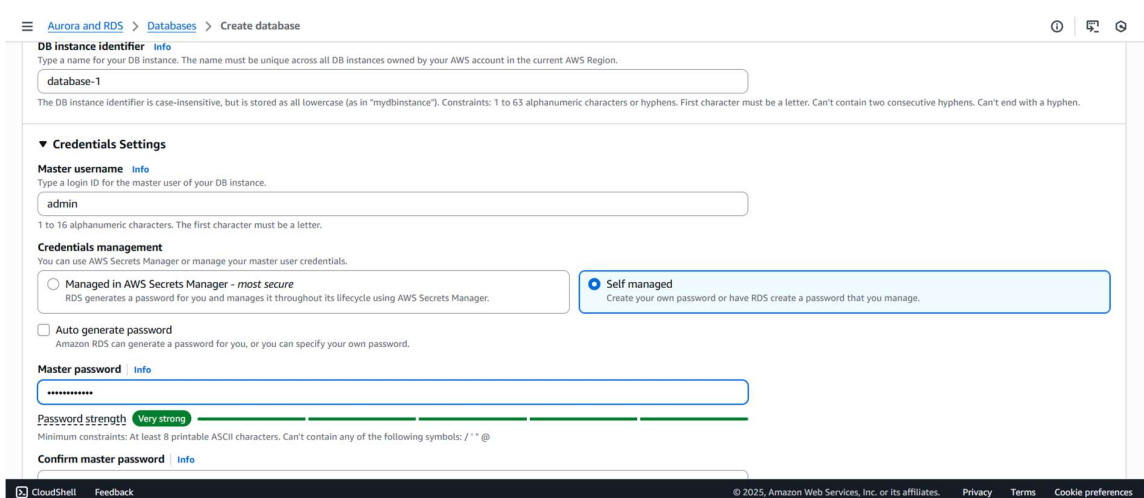
Steps to launch MariaDB RDS Instance:

- Log in to AWS, select **Aurora and RDS**
- Click on Database and click on **“Create Database.”**



The screenshot shows the AWS 'Create database' console page. At the top, there's a navigation bar with 'Aurora and RDS' and 'Databases' tabs. Below the navigation bar, a banner states: 'Free plan has access to limited features and resources. The free plan limits the features and resources that are available for RDS and Aurora databases. Upgrade your account plan to remove all limitations. Learn more'. The main section is titled 'Create database' and includes an 'Info' link. Under 'Choose a database creation method', 'Standard create' is selected, with a description: 'You set all of the configuration options, including ones for availability, security, backups, and maintenance.' Below this, the 'Engine options' section is visible, showing several database engine choices: Aurora (MySQL Compatible), Aurora (PostgreSQL Compatible), MySQL, PostgreSQL, MariaDB (selected), and Oracle. Each option has a corresponding icon and a brief description.

- Select standard, click on MariaDB



The screenshot shows the 'DB instance identifier' and 'Credentials Settings' sections of the 'Create database' console. The 'DB instance identifier' section has a text input field with 'database-1' entered. Below this, the 'Credentials Settings' section is visible, showing 'Self managed' selected for 'Managed in AWS Secrets Manager - most secure'. Below this, there's a 'Master password' field with a strength indicator showing 'Very strong'. The 'Master password' field has a minimum constraint of 8 printable ASCII characters and cannot contain any of the following symbols: / ' * @.

- Select Template, click on **“Provide Details.”**
- Select **Instance and Storage.**

Connectivity info

Compute resource
Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ **Don't connect to an EC2 compute resource**
Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ **Connect to an EC2 compute resource**
Set up a connection to an EC2 compute resource for this database.

Network type info
To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.

☒ **IPv4**
Your resources can communicate only over the IPv4 addressing protocol.

☐ **Dual-stack mode**
Your resources can communicate over IPv4, IPv6, or both.

Virtual private cloud (VPC) info
Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

Default VPC (vpc-017a52f7ad1bbfb1b)
6 Subnets, 6 Availability Zones

Only VPCs with a corresponding DB subnet group are listed.

ⓘ After a database is created, you can't change its VPC.

DB subnet group info
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Configure Authentication and monitoring settings and click **Create**.

Monitoring info

Choose monitoring tools for this database. Database Insights provides a combined view of Performance Insights and Enhanced Monitoring for your fleet of databases. Database Insights pricing is separate from RDS monthly estimates. See [Amazon CloudWatch pricing](#).

☐ Database Insights - Advanced

- Retains 15 months of performance history
- Fleet-level monitoring
- Integration with CloudWatch Application Signals

☒ **Database Insights - Standard**

▼ **Additional monitoring settings**
Enhanced Monitoring, CloudWatch Logs and DevOps Guru

Enhanced Monitoring

☒ **Enable Enhanced monitoring**
Enabling Enhanced Monitoring metrics are useful when you want to see how different processes or threads use the CPU.

OS metrics granularity

60 seconds

Monitoring role for OS metrics

default

The monitoring role is an IAM role that allows RDS to send Enhanced Monitoring metrics to Amazon CloudWatch Logs. Choose an existing monitoring role, or choose **default** to have RDS automatically create the IAM role `rds-monitoring-role` for you.

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Aurora and RDS

Dashboard

Databases

Performance insights

Snapshots

Exports in Amazon S3

Automated backups

Reserved instances

Proxies

Subnet groups

Parameter groups

Option groups

Custom engine versions

Zero-ETL integrations

Events

Event subscriptions

Creating database database-1
Your database might take a few minutes to launch. You can use settings from database-1 to simplify configuration of suggested database add-ons while we finish creating your DB for you. [View connection details](#)

Databases (1) ☒ Group resources [Modify](#) [Actions](#) [Create database](#)

Filter by databases

DB identifier	Status	Role	Engine	Region ...	Size
database-1	Creating	Instance	MariaDB	us-east-1a	db.t4g.micro

5. Migrate the database from EC2 to RDS.

Purpose: This task involves moving your MariaDB database from a self-managed EC2 instance to Amazon RDS, AWS's fully managed database service. The main purpose is to eliminate manual administrative overhead—such as backups, patching, and scaling—while improving reliability, performance, and security. Migrating to RDS ensures high availability, automated backups, and seamless scalability, allowing you to focus on application development rather than database maintenance.

Steps to Migrate the Data from EC2 to RDS:

Step 1: Take a dump of the Database.

- Using the command `mysqldump -u root -p database_name > file_name.sql`
- Migrate the DB dump to RDS using the following commands:
 - `mysql -h endpoint -p 3306 -u user_name -p RDS_database_name`
 - `show databases;`
 - `create database database_name;`
 - `exit`
 - `mysql -h endpoint -p 3306 -u user_name -p RDS_database_name < file_name.sql`
 - `mysql -h endpoint -p 3306 -u user_name -p RDS_database_name`
 - `show database;`
 - `use database_name;`
 - `select * from table1;`

6. Install MySQL DB on EC2.

Purpose: This task involves setting up a MySQL database server on an Amazon EC2 instance. The main purpose is to create a self-managed, flexible, and customizable relational database environment for development, testing, or production use. By installing MySQL on EC2, you gain full control over database configurations, security settings, and performance optimizations, making it ideal for applications requiring tailored database solutions.

Steps to Install MySQL Database on EC2:

Step 1: Remove Mariadb using the following commands

- `# sudo dnf remove -y mariadb105*`
- `# sudo dnf clean packages`

```
root@ip-172-31-17-251 ec2-user]# sudo dnf remove -y mariadb105*
dependencies resolved.
```

Package	Architecture	Version	Repository	Size
removing:				
mariadb105	x86_64	3:10.5.29-1.amzn2023.0.1	@amazonlinux	18 M
mariadb105-backup	x86_64	3:10.5.29-1.amzn2023.0.1	@amazonlinux	23 M
mariadb105-common	x86_64	3:10.5.29-1.amzn2023.0.1	@amazonlinux	179 k
mariadb105-cracklib-password-check	x86_64	3:10.5.29-1.amzn2023.0.1	@amazonlinux	16 k
mariadb105-errmsg	x86_64	3:10.5.29-1.amzn2023.0.1	@amazonlinux	2.6 M
mariadb105-gssapi-server	x86_64	3:10.5.29-1.amzn2023.0.1	@amazonlinux	19 k
mariadb105-server	x86_64	3:10.5.29-1.amzn2023.0.1	@amazonlinux	68 M
mariadb105-server-utils	x86_64	3:10.5.29-1.amzn2023.0.1	@amazonlinux	798 k
removing unused dependencies:				
mariadb-connector-c	x86_64	3.3.10-1.amzn2023.0.1	@amazonlinux	584 k
mariadb-connector-c-config	noarch	3.3.10-1.amzn2023.0.1	@amazonlinux	497
mysql-selinux	noarch	1.0.4-2.amzn2023.0.3	@amazonlinux	49 k
perl-B	x86_64	1.80-477.amzn2023.0.7	@amazonlinux	491 k
perl-DBD-MariaDB	x86_64	1.22-1.amzn2023.0.4	@amazonlinux	378 k
perl-DBI	x86_64	1.643-7.amzn2023.0.3	@amazonlinux	1.9 M
perl-Data-Dumper	x86_64	2.174-460.amzn2023.0.2	@amazonlinux	109 k

Step 2: Go to the “/opt” directory and run the following commands to download and install MySQL.

- `# sudo wget https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm`
- `# sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y`
- `# sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023`
- `# sudo dnf install mysql-community-server -y`

```
[root@ip-172-31-17-251 opt]# sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y
Last metadata expiration check: 0:51:22 ago on Mon Oct 13 09:23:55 2025.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
mysql80-community-release	noarch	el9-1	@commandline	10 k

```
Transaction Summary
=====
Install 1 Package

Total size: 10 k
Installed size: 5.7 k
Downloading Packages:
Running transaction check
Transaction check succeeded
```

- *# sudo systemctl start mysqld*
- *# sudo systemctl enable mysqld*
- *# sudo mysql_secure_installation*
- *# sudo systemctl status mysqld*

```
complete!
[root@ip-172-31-17-251 opt]# sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023
[root@ip-172-31-17-251 opt]# sudo dnf install mysql-community-server -y
MySQL 8.0 Community Server                    54 MB/s | 2.7 MB    00:00
MySQL Connectors Community                    6.2 MB/s | 90 kB    00:00
MySQL Tools Community                         43 MB/s | 1.2 MB    00:00
dependencies resolved.
```

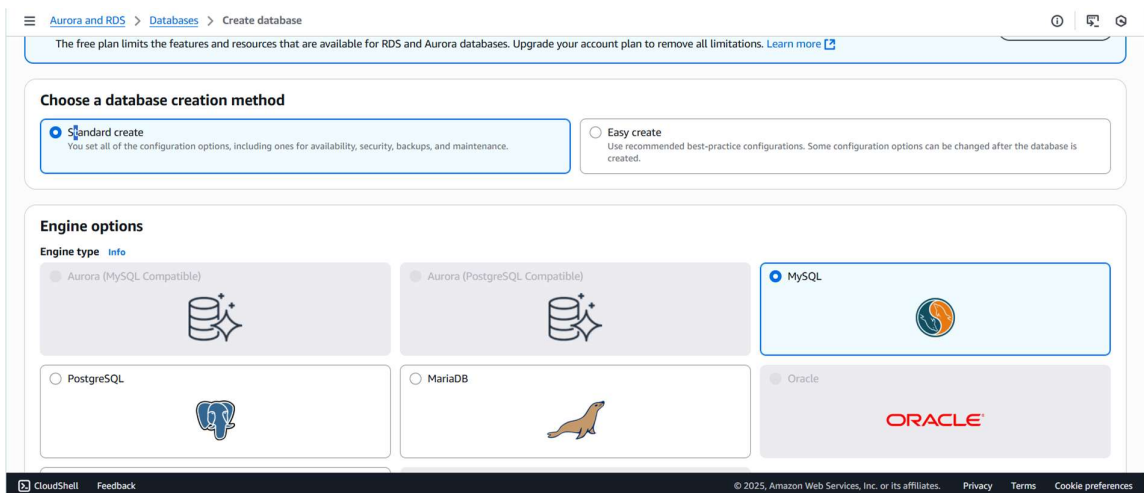
Package	Architecture	Version	Repository	Size
Installing:				
mysql-community-server	x86_64	8.0.43-1.el9	mysql80-community	50
Installing dependencies:				
mysql-community-client	x86_64	8.0.43-1.el9	mysql80-community	3.3
mysql-community-client-plugins	x86_64	8.0.43-1.el9	mysql80-community	1.4
mysql-community-common	x86_64	8.0.43-1.el9	mysql80-community	556
mysql-community-icu-data-files	x86_64	8.0.43-1.el9	mysql80-community	2.3
mysql-community-libs	x86_64	8.0.43-1.el9	mysql80-community	1.5

7. Launch MySQL RDS image.

Purpose: This task involves creating a fully managed MySQL database instance using Amazon RDS (Relational Database Service). The main purpose is to eliminate manual database administration—such as backups, patching, and scaling—while ensuring high availability, security, and performance. With RDS, you can focus on application development, as AWS handles routine maintenance, automated backups, and failover management.

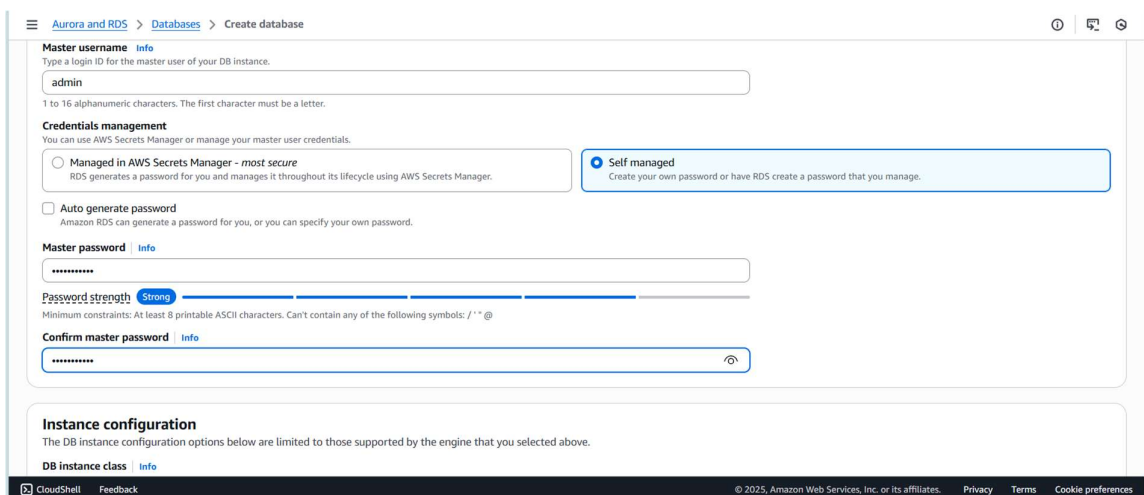
Steps to launch MySQL RDS Image:

- Login to AWS, select Aurora and RDS
- Click on Database and click on **“Create Database.”**



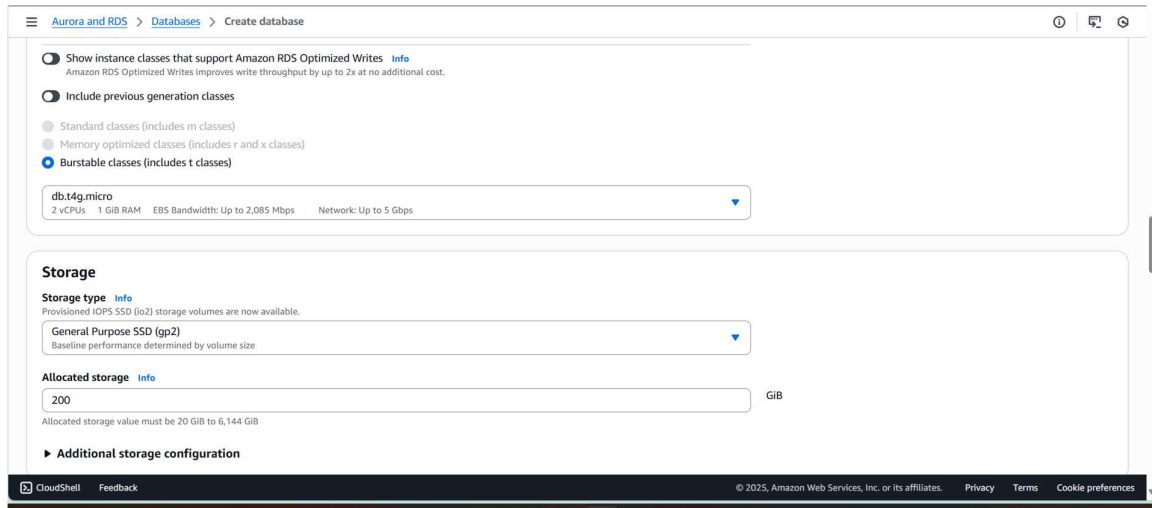
The screenshot shows the AWS 'Create database' console. The breadcrumb navigation is 'Aurora and RDS > Databases > Create database'. A notice at the top states: 'The free plan limits the features and resources that are available for RDS and Aurora databases. Upgrade your account plan to remove all limitations. [Learn more](#)'. Under 'Choose a database creation method', 'Standard create' is selected. In the 'Engine options' section, 'MySQL' is selected under 'Engine type'. Other options include Aurora (MySQL Compatible), Aurora (PostgreSQL Compatible), PostgreSQL, MariaDB, and Oracle. The footer includes 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc.

- Select standard create
- Select MySQL



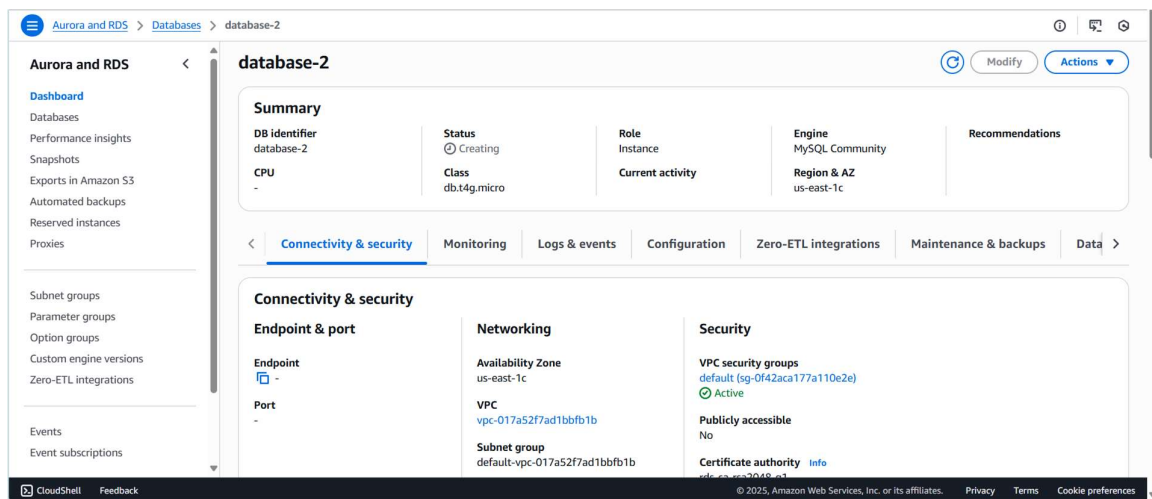
The screenshot shows the 'Master username' and 'Credentials management' sections of the 'Create database' console. The 'Master username' field contains 'admin'. Under 'Credentials management', 'Self managed' is selected. The 'Master password' field is empty, and the 'Password strength' indicator shows 'Strong'. The 'Confirm master password' field is also empty. The 'Instance configuration' section is partially visible at the bottom. The footer includes 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc.

- Enter Master username & password and select instance and storage.



The screenshot shows the 'Create database' page in the Amazon RDS console. The breadcrumb trail is 'Aurora and RDS > Databases > Create database'. There are three tabs: 'Show instance classes that support Amazon RDS Optimized Writes', 'Include previous generation classes', and 'Burstable classes (includes t classes)'. The 'Burstable classes' tab is selected, and a dropdown menu shows 'db.t4g.micro' as the selected instance class. Below this, the 'Storage' section is visible, showing 'Storage type' as 'General Purpose SSD (gp2)' and 'Allocated storage' as '200 GIB'. The footer of the console shows 'CloudShell', 'Feedback', and copyright information for Amazon Web Services.

- Select connectivity settings and click create.



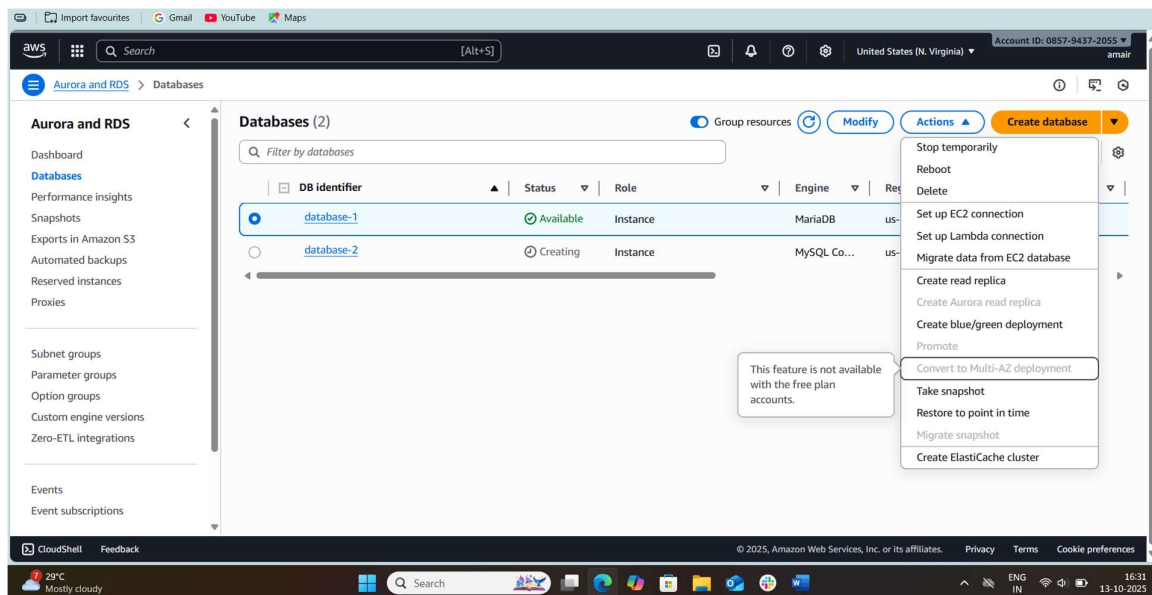
The screenshot shows the 'database-2' page in the Amazon RDS console. The breadcrumb trail is 'Aurora and RDS > Databases > database-2'. The left sidebar shows the 'Aurora and RDS' navigation menu. The main content area has a 'Summary' section with details like 'DB identifier: database-2', 'Status: Creating', 'Role: Instance', 'Engine: MySQL Community', and 'Region & AZ: us-east-1c'. Below the summary, there are tabs for 'Connectivity & security', 'Monitoring', 'Logs & events', 'Configuration', 'Zero-ETL integrations', 'Maintenance & backups', and 'Data'. The 'Connectivity & security' tab is selected, showing 'Endpoint & port' (Endpoint: -, Port: -), 'Networking' (Availability Zone: us-east-1c, VPC: vpc-017a52f7ad1bbfb1b, Subnet group: default-vpc-017a52f7ad1bbfb1b), and 'Security' (VPC security groups: default (sg-0f42aca177a110e2e), Publicly accessible: No, Certificate authority: info). The footer of the console shows 'CloudShell', 'Feedback', and copyright information for Amazon Web Services.

8. Configure Multi AZ.

Purpose: This task involves enabling Multi-AZ deployment for your MySQL RDS instance. The main purpose is to enhance database availability and durability by automatically replicating your primary database to a standby instance in a different Availability Zone (AZ). In the event of a failure, Amazon RDS automatically fails over to the standby instance, minimizing downtime and ensuring continuous operation.

Steps to configure Multi AZ:

- Go to **RDS**, select **Database**
- Select **Database**, click on **Actions**
- Convert to **multi-AZ deployment**.

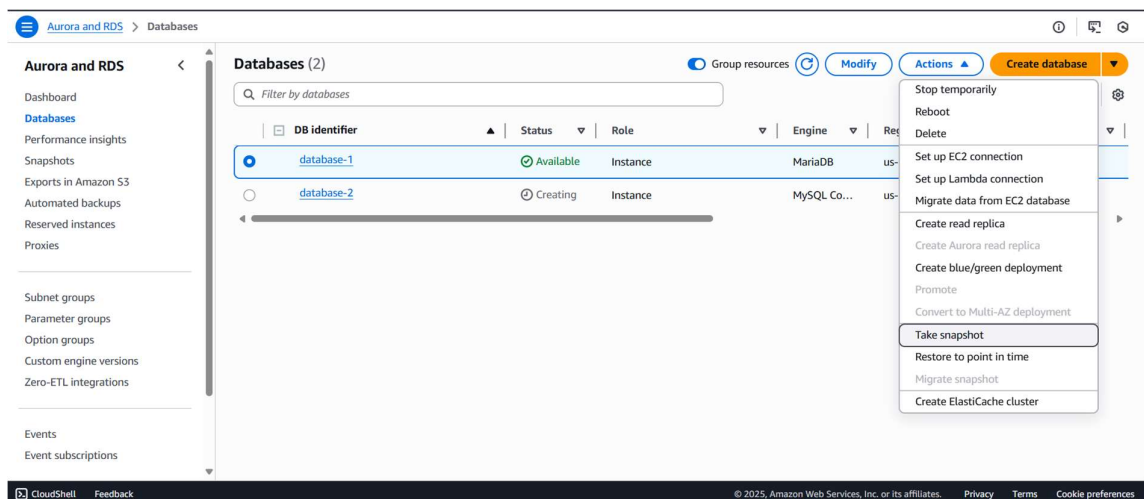


9. Take a backup of the DB and restore the DB.

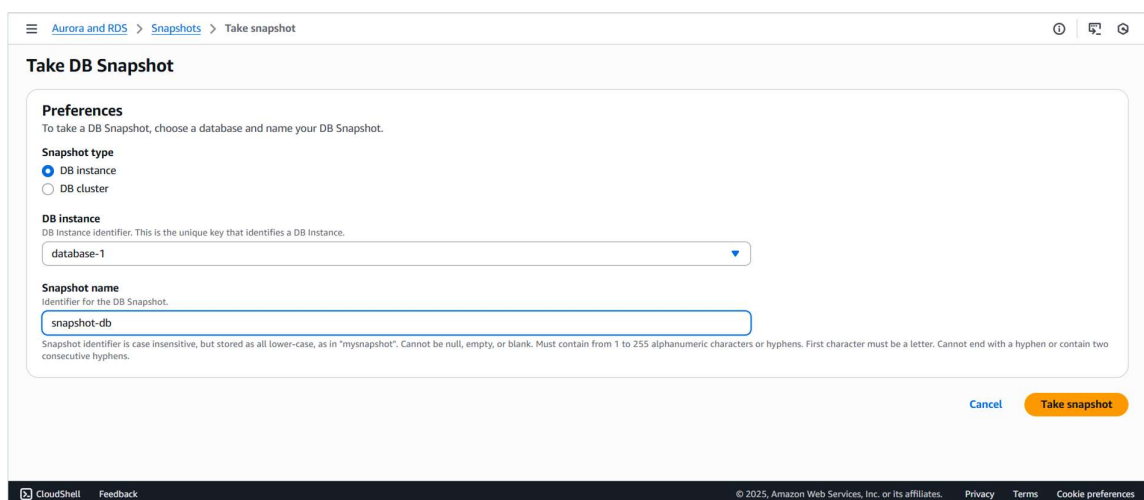
Purpose: This task involves creating a backup of your MySQL database (either on EC2 or RDS) and restoring it when needed. The main purpose is to ensure data protection and quick recovery in case of accidental deletion, corruption, or system failures. For RDS, you can use automated snapshots or manual backups, while for EC2, you can use tools like mysqldump or xtrabackup. Restoring the backup allows you to recover your database to a previous state, minimizing downtime and data loss.

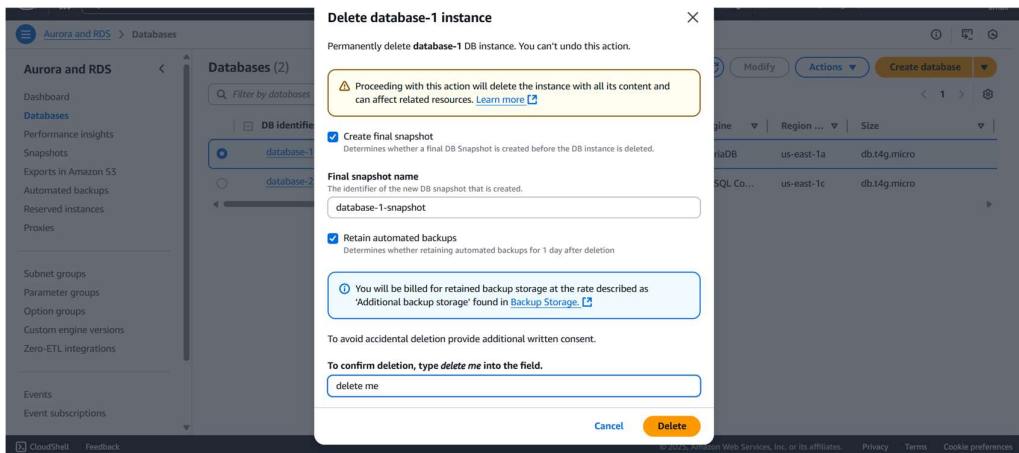
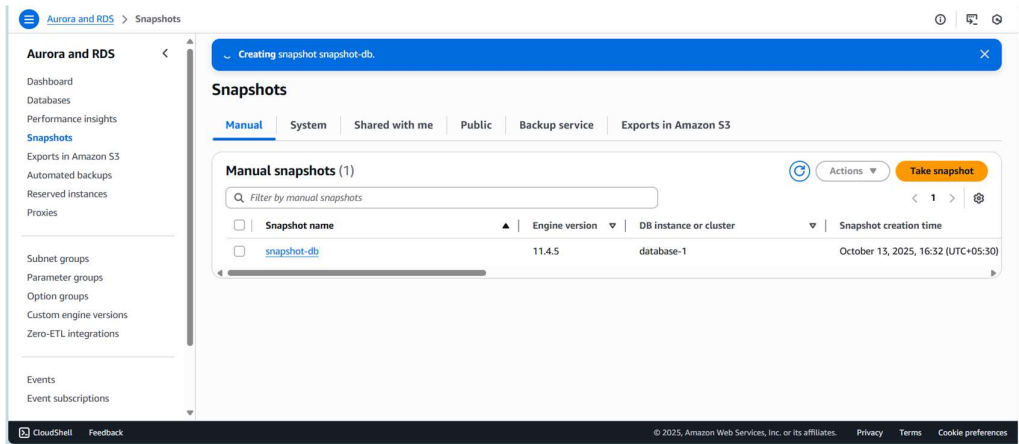
Steps to take a backup and restore the DB:

- Go to the RDS
- Click on action and click on the snapshot

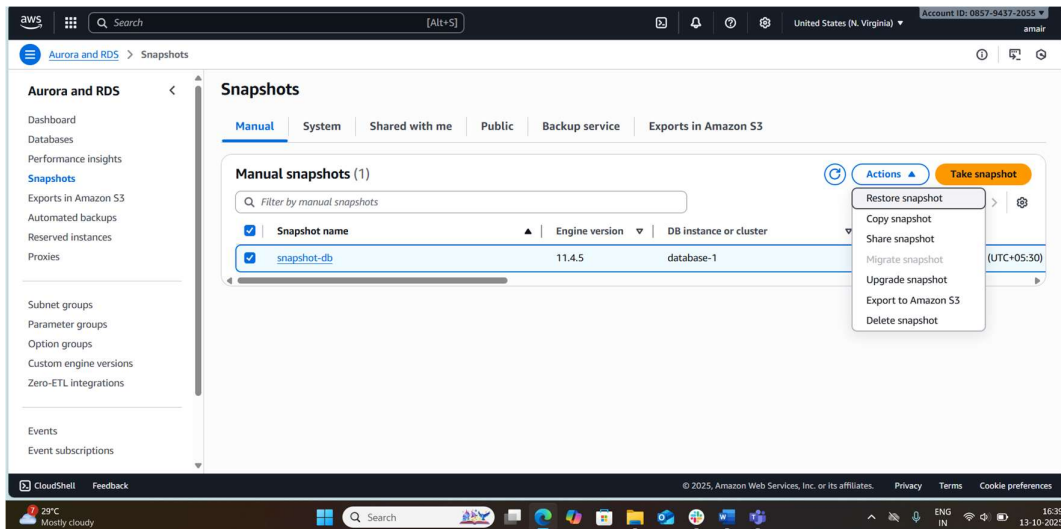


- After taking a snapshot, delete the database.
- Go to RDS, select snapshots





- Select your snapshot and go to Actions
- Click on restore snapshot.

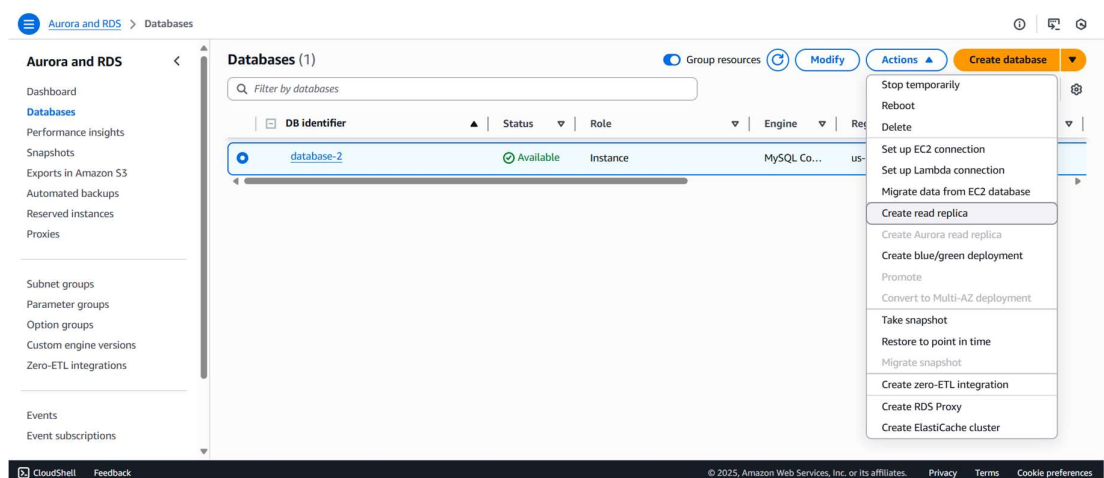


10. Create a Read Replica.

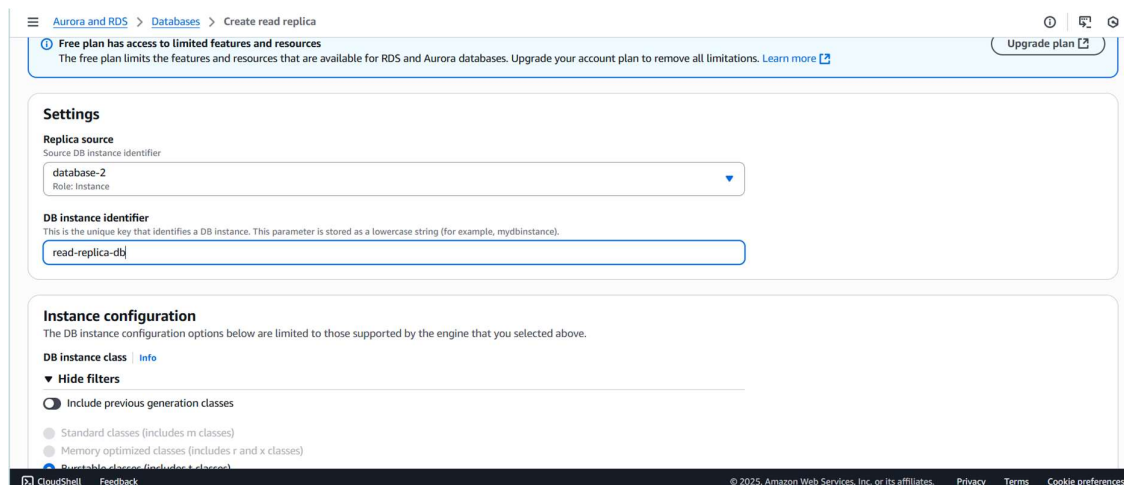
Purpose: This task involves setting up a Read Replica for your MySQL RDS database instance. The main purpose is to offload read-heavy workloads from your primary database, improving performance and scalability for applications with high read traffic. A Read Replica asynchronously replicates data from the primary instance, allowing you to distribute read queries across multiple replicas. This enhances fault tolerance, reduces latency, and supports disaster recovery by providing a redundant copy of your data.

Steps to create a Read Replica:

- Select your primary DB instance.
- Click Actions → Create read replica.



- Fill in the details:
 - DB Instance Identifier: Give a name for the replica
 - DB Instance Class: Choose based on your read workload.
 - Multi-AZ Deployment: Optional (for HA).



- Click **Create read replica**.

The screenshot shows the 'Create read replica' page in the AWS Management Console. The breadcrumb navigation is 'Aurora and RDS > Databases > Create read replica'. The page has a sidebar on the left with a menu icon and a search bar. The main content area contains several sections:

- Choose VPC security groups:** A dropdown menu with 'default' selected.
- Availability Zone:** A dropdown menu with 'No preference' selected. Below it, a note states: 'The EC2 Availability Zone that the database will be created in.'
- Certificate authority - optional:** A dropdown menu with 'rds-ca-rsa2048-g1 (default)' selected. Below it, a note states: 'Using a server certificate provides an extra layer of security by validating that the connection is being made to an Amazon database. It does so by checking the server certificate that is automatically installed on all databases that you provision. Expiry: May 26, 2061. If you don't select a certificate authority, RDS chooses one for you.'
- Additional configuration:** A section with a right-pointing arrow.
- Database authentication:** A section with a right-pointing arrow. It contains a sub-section 'Database authentication options' with three radio buttons:
 - Password authentication:** Selected. Description: 'Authenticates using database passwords.'
 - Password and IAM database authentication:** Unselected. Description: 'Authenticates using the database password and user credentials through AWS IAM users and roles.'
 - Password and Kerberos authentication:** Unselected.

The footer of the console shows 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc. or its affiliates.

- Wait for Replica to be Available
- The read replica will show as creating.
- Once ready, the status will change to available

The screenshot shows the 'Databases' page in the AWS Management Console. The breadcrumb navigation is 'Aurora and RDS > Databases'. The page has a sidebar on the left with a menu icon and a search bar. The main content area contains a blue banner at the top with the text: 'Creating replica read-replica-db in US East (N. Virginia). Your database might take a few minutes to launch.' Below the banner, there is a section 'Databases (2)' with a search bar and a table of databases.

DB identifier	Status	Role	Engine	Region ...	Size
database-2	Modify...	Primary	MySQL Co...	us-east-1c	db.t4g.micro
read-replica-db	Creating	Replica	MySQL Co...	-	db.t4g.micro

The footer of the console shows 'CloudShell', 'Feedback', and copyright information for Amazon Web Services, Inc. or its affiliates.